

I've Been a Costco Member for 25 Years. Last Month I Built an AI Agent to Get My Money Back

The whole thing runs serverless for under a dollar a month.



Background

I've been a Costco member since 2001. That's 25 years of bulk toilet paper, ziplock bags, and the occasional impulse buy of a 48-pack of batteries I definitely didn't need.

Over those years, one thing has kept me loyal: their customer service. It's genuinely good. It reminds me of Amazon's Customer Obsession Leadership Principle, where the customer is the starting point and you work backwards from there. Costco lives that.

The Pattern I Couldn't Ignore

Here's something that kept happening to me. I'd buy a jacket for \$24.99 on a Tuesday. The following week, I'd walk past the same jacket with a new price tag: \$5.00 off. I'd buy a bag of dumplings for \$14.99. Two weeks later, on sale for \$11.99. This happened over and over. I started to wonder if Costco was personally timing their sales around my shopping trips.

Costco has a price adjustment policy. If something you bought goes on sale within 30 days, you can go to the membership counter and get the difference back. The problem is you have to know it went on sale. You have to remember what you paid. You have to dig up the receipt. And you have to actually go do it before the 30 days are up.

I never did any of that. I just ate the difference every time.

So I Built a Thing

I'm a builder. When I see a repetitive problem, I want to automate it. So I built an AI-powered Costco Receipt Scanner & Price Match Agent that does the whole thing for me. Completely Serverless with Amazon Bedrock AgentCore!

Here's what it does:

1. I upload my receipt PDFs (from my phone, from email, wherever)
2. Amazon Nova AI parses every line item, price, item number, and whether I already got a temporary price drop
3. Scrapers pull current deals from the web and Costco coupon book
4. An AI agent cross-references my purchases against every active deal and tells me exactly which items dropped in price, how much I'd save, and what to say at the counter

The whole thing runs on AWS. FastAPI backend, DynamoDB for storage, S3 for receipt PDFs, Bedrock for the AI, and a web UI I can hit from my laptop or phone.

The Weekly Agent

I didn't want to manually check every week. So I built and deployed an Amazon Bedrock AgentCore agent with Strands SDK that runs automatically every Friday at 9 PM. It scrapes all the deal sources, runs the analysis across every receipt I've uploaded, and emails me a formatted HTML report via SES.

The report has two sections. The first table shows price adjustment opportunities: items I paid full price for that are now on sale, with the exact savings per item. The second table shows items where Costco already applied a Temporary Price Drop (TPD) at checkout, so I can see what I saved without doing anything. Every item links back to the actual deal post so I can verify the prices myself, and every receipt reference is a presigned S3 link to the original PDF.

Friday night I get an email. Saturday morning I walk into Costco with my receipts. Done.

The first time it ran, it found \$9 in price adjustments I would have missed and confirmed \$18 in TPD savings I didn't even know I'd gotten. Not life-changing money, but it adds up when it happens every week.

The Architecture

For anyone who wants to look under the hood:

- **Frontend:** Single HTML file hosted on AWS Amplify, with Cognito authentication. Same file works locally during development with no auth. Environment-aware through a config.js that gets generated at deploy time.
- **API:** API Gateway HTTP API with Cognito JWT authorizer, backed by a Lambda function running the FastAPI app via Mangum. The analysis endpoint streams results back to the browser in real time so you can watch the AI think through each item match as it happens.
- **AI:** Amazon Nova models for receipt parsing (OCR) and price match analysis. Nova 2 Lite for standard parsing, Nova Premier for complex receipts that need a second pass.
- **Automation:** AgentCore Runtime runs the weekly agent inside a container. EventBridge Scheduler triggers it using a universal target (`aws-sdk:bedrockagentcore:invokeAgentRuntime`) that calls AgentCore directly with zero Lambda functions in between. The agent scans deals, runs analysis, converts the markdown report to inline-styled HTML, and sends it through SES. The SES email identity is created by CDK on first deploy and sends a verification email automatically. One schedule, one container, one email.
- **Storage:** DynamoDB for receipts and deals, S3 for receipt PDFs with presigned URLs for secure access from emails.
- **Infrastructure:** Everything defined in CDK (TypeScript) across three stacks. Repeatable, immutable, deploy to any region.

The flow for the weekly scan looks like this:

One thing worth calling out: EventBridge Scheduler's universal target for AgentCore is undocumented as of this writing. I found it by looking at the supported SDK targets and guessing the ARN format. It works, but the scheduler reports the invocation as failed even when it succeeds (the streaming response confuses its success detection). Setting retries to zero prevents duplicate emails. I learned this the hard way.

What It Costs

Here's what the project cost me in us-west-2 over the first two weeks (Feb 1–13), including all development and testing:

Service	Cost	Notes
Amazon Bedrock (Nova)	\$0.47	Receipt parsing + price match analysis
Amazon Bedrock AgentCore	\$0.23	Container runtime for weekly agent
Amazon DynamoDB	\$0.02	Receipts + deals tables
Amazon ECR	\$0.01	Docker image storage
API Gateway	\$0.00	Free tier
AWS Amplify	\$0.00	Free tier
AWS Lambda	\$0.00	Free tier
Amazon SES	\$0.00	Free tier
Amazon S3	\$0.00	Free tier
Total	\$0.73	

Under a dollar for two weeks of building and testing. In steady state (one weekly scan, a few receipt uploads), expect roughly \$0.10–0.20/week. Lambda, SES, DynamoDB, API Gateway, and Amplify all fall within free tier for personal use. Bedrock has no free tier, but Nova 2 Lite is cheap at \$0.32 per million input tokens.

What I Learned

Most of this project was written with [Kiro CLI](#), Amazon's AI coding assistant. What would have taken weeks of wiring up CDK stacks, debugging IAM policies, and figuring out EventBridge Scheduler's undocumented universal targets was done in a few hours of back-and-forth. I described what I wanted, Kiro wrote the code, I tested it, we iterated. The entire CDK infrastructure, the agent, the streaming API, the email converter, all of it.

A few things that tripped us up along the way:

- Costco receipt abbreviations are wild. “CKN/VEG DUMP” is dumplings. “T TURTLENECK” is a sweater. Nova handles it well, but I added a two-tier system where Nova Premier reparses anything Nova Lite struggles with.
- Deduplication matters. Two of the same item should both show up, but each should only match the best deal. Tracking by receipt position solved this.
- TPD (Temporary Price Drop) detection was a late addition that turned out to be one of the most useful features. Costco applies these at checkout automatically. The scanner picks them up and shows what you saved without asking.

Deploy It Yourself

Everything deploys through CDK. Three stacks, one command.

Prerequisites

- AWS CLI configured with credentials
- Node.js 18+ and npm
- Docker running (for Lambda and AgentCore container builds)
- Python 3.12+
- A verified email address in Amazon SES (the region you're deploying to)

Run Locally

You can run the whole thing on your laptop without deploying anything except the DynamoDB tables and S3 bucket (which the Common stack creates).

```
# Create a virtual environment and install dependencies
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Run the app
./run.sh
```

This starts FastAPI on `http://localhost:8000`. The web UI at `static/index.html` works locally without Cognito auth. Just open the HTML file in your browser and point it at localhost. The `run.sh` script sets the default env vars (region, table names, bucket) so you don't need to configure anything if you deployed the Common stack with defaults.

Deploy

```
# Clone and install CDK dependencies
cd costco-scanner/infra
npm install
# Deploy everything (Lambda, Amplify, API Gateway, Cognito, DynamoDB, S3)
# notifyEmail: your email address for weekly reports (SES verification sent on first deploy)
cd .. && ./deploy.sh
# Deploy the weekly agent (optional, requires a verified SES email)
cd infra && npx cdk deploy CostcoScannerAgentCore \
  -c region=us-west-2 \
  -c notifyEmail=your-email@example.com \
  --require-approval never
```

The `region` context parameter controls where everything goes. The `notifyEmail` parameter is the email that receives the weekly report from the AgentCore agent. On first deploy, CDK creates an SES identity for that email and sends a verification link. Click it once and the weekly reports start flowing.

After deploy, CDK outputs the Amplify URL, API endpoint, and Cognito pool details. The AgentCore stack also creates an SES email identity for the `notifyEmail` address, which triggers a verification email. Click the link in that email, and the weekly reports start working. Subsequent deploys won't re-trigger verification. Create a Cognito user through the AWS console or CLI, then log in to the web UI.

Cleanup

```
# Destroy the AgentCore stack first (it depends on Common)
cd infra && npx cdk destroy CostcoScannerAgentCore -c region=us-west-2 -c notifyEmail=your-email@example.com

# Destroy Amplify + API stack
npx cdk destroy CostcoScannerAmplify -c region=us-west-2
# Destroy shared resources (DynamoDB, S3, ECR) last
npx cdk destroy CostcoScannerCommon -c region=us-west-2
```

S3 buckets and DynamoDB tables have `RemovalPolicy.DESTROY` set, so they'll clean up with the stack. ECR repos too. If you want to keep your receipt data, back up the DynamoDB tables before destroying.

Open Source

I'm making this open source. The code is available on [GitHub](#). If you're a Costco member who's tired of leaving money on the table, clone it, deploy it, and start getting your price adjustments.

The CDK stacks mean you can deploy the entire infrastructure to your own AWS account with a single command. Upload your receipts, let the agent do its thing, and walk into Costco knowing exactly what you're owed.

If you're a developer who wants to learn how to build AI agents on AWS, this is a real working example. Not a toy demo. Not a chatbot that tells you the weather. A thing that actually saves you money every week.

What's Next

I'm building a native iOS app using Amplify Library for Swift, so I can snap a photo of my receipt right at the checkout and upload it on the spot. Same Cognito auth, same API, same backend. Just a better experience on the phone.

25 years of Costco membership. Thousands of dollars in missed price adjustments. One AI agent to make sure that never

happens again.

The hot dog combo is still \$1.50 and the rotisserie chicken is still \$7.99. Some things never change.

